# Starter

Match the following processes to the relevant TCP/IP Layers (Application, Transport, Network, Link)

1. Splits up a message into packets.
2. Provides the interface and protocols needed by the user.
3. If it does not receive an ACK for a particular packet, will resend.
4. Sends an ACK when it receives a packet.
5. Controls physical connection between hardware devices using MAC addresses.
6. Adds the IP Address of the source and destination.
7. Adds the port number of the source and destination.
8. Adds the sequence number of a packet.

# SA 3, January

Last lesson in the first week back after Christmas (five weeks away).

Potential topics:

- **Theory only (no C# programming)**
- Everything covered up until the Christmas holidays (see BPCompSci for full list of topics we've covered)

# Application Layer Protocols

Keywords: Port; Socket; SSH; FTP; SMTP; POP3; HTTP; HTTPS

# Recap

1. Define the term **socket**
2. Write down as many Protocols as you can remember from last lesson.
3. State the difference between an IP Address and a socket.
4. State the difference between an IP Address and a MAC Address.

# Recap

1. Define the term **socket**

The combination of an IP Address and Port Number,
eg: 192.4.1.5 : 80

# Recap

2. Write down as many Protocols as you can remember from last lesson.

HTTP; HTTPS; POP(3); IMAP; SMTP; FTP; SSH;

# Recap

3. State the difference between an IP Address and a socket.

A socket is the combination of an IP address **and** a port number.

# Recap

4. State the difference between an IP Address and a MAC Address.

An IP address is an address for a host on the internet. IPv4 addresses are not necessarily unique

A MAC address is a unique address for a device used for direct communication (used for single hops)

# Explain to your peers!

Your group will be given a standard protocol.
You have 5 minutes to discuss the best way for you to describe it to the rest of the class.
Then you will take turns to describe your protocol to the class.

**Include**:

- What it stands for
- What is its purpose
- What is the port number

# Presentations

Lock your computers down for this!

## When listening:

- No notes to be taken
- Give your full attention to the speakers

## After the presentation:

- Clap.
- Write down a summary of what they have said
- Put your hand up to ask for clarification or a question

# Summary:

**HTTP** – port 80

- Hypertext Transfer Protocol
- Viewing web pages

**HTTPS** – port 443

- Secure HyperText Transfer Protocol
- Accessing secure web pages

**SSH** – port 22

- Secure shell – used for remotely accessing and managing a computer. Uses public key encryption.

**Post Office Protocol** (POP3) – port 110

- Downloading personal emails from a web server

**SMTP** – port 25

- Simple Mail Transfer Protocol
- Sending email messages

**FTP** – port 21 (and 20)

- File Transfer Protocol
- Uploading files to a web server

# “Exam”ple question

On Google classroom,
Attempt it **without** your notes:

## L131 – Networks exam question

You must attempt this question on paper as it is from a Paper 2.

Only attempt on the computer if you are entitled to this as an access arrangement.

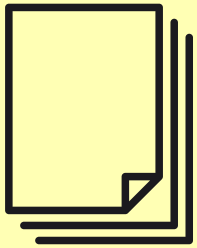
# Quizlet!

You'll be split into teams, and **some** of you will have the right answer to a question.

Don't guess, because you'll be set back if you get it wrong as a **team**.

Quizlet Link

# Structures - RECAP



- Structures are like containers
- They group together a set of variables that are related to create a **record**
- Structures are **statically allocated**

```
struct Book {
  public string title;
  public string author;
  public string genre;
  public decimal price;
}
```

This is the template for a Book structure.

# Example - RECAP

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

# Example - RECAP

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

# Example - RECAP

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

Then you can declare variables of that type.

# Example - RECAP

```
struct Student {
  public string name;
  public int age;
}
static void Main(string[] args) {
  Student FavouritePerson;

  Console.WriteLine("Enter name:");
  FavouritePerson.name = Console.ReadLine();
  Console.WriteLine("Enter an age:");
  FavouritePerson.age = int.Parse(Console.ReadLine());

  Console.WriteLine(FavPerson.name + "is the best, and only
"+FavPerson.age);
}
```

Structures define templates

Then you can declare variables of that type.

You access the data with the '.'

# Arrays of Structures

# Discuss

How are arrays and structures the same?

How are they different?

Come up with at least 3 points for each.

# Making an array of structs

```
struct StudentRecord {
  public string name;
  public int score;
}

static void Main(string[] args) {
  StudentRecord[] TestScores = new StudentRecord[2];

  TestScores[0].name = "Isabel";
  TestScores[0].score = 100;
  TestScores[1].name = "Mr Tall";
  TestScores[1].score = 78;

  for (int i = 0; i < 2; i++)
  {
    Console.WriteLine(TestScores[i].name + " scored " + TestScores[i].score);
  }
  Console.ReadKey();
}
```

# Arrays of Structures

`StudentRecord[]` `StudentTest`

| | |
|---|---|
| Name: Mr Jelly<br>Score: 3 | 0 |
| Name: Mr Small<br>Score: 87 | 1 |
| Name: Mr Strong<br>Score: 2 | 2 |
| Name: Mr Tall<br>Score: 56 | 3 |
| Name: Mr Nosey<br>Score: 33 | 4 |

```
struct StudentRecord {
  public string name;
  public int score;
}
```

```
studentTest[2].Name =
studentTest[1].Score =
studentTest[0].Name =
```

# Arrays of Structures

`StudentRecord[]` `StudentTest`

| | |
|---|---|
| Name: Mr Jelly<br>Score: 3 | 0 |
| Name: Mr Small<br>Score: 87 | 1 |
| Name: Mr Strong<br>Score: 2 | 2 |
| Name: Mr Tall<br>Score: 56 | 3 |
| Name: Mr Nosey<br>Score: 33 | 4 |

```
struct StudentRecord {
  public string name;
  public int score;
}
```

```
studentTest[2].Name =    Mr Strong
studentTest[1].Score =   87
studentTest[0].Name =    Mr Jelly
```

# Arrays of Structures

`StudentRecord[]` `StudentTest`

| | |
|---|---|
| Name: Mr Jelly<br>Score: 3 | 0 |
| Name: Mr Small<br>Score: 87 | 1 |
| Name: Mr Strong<br>Score: 2 | 2 |
| Name: Mr Tall<br>Score: 56 | 3 |
| Name: Mr Nosey<br>Score: 33 | 4 |

```
struct StudentRecord {
  public string name;
  public int score;
}
```

`studentTest[2].Name =` Mr Strong
`studentTest[1].Score =` 87
`studentTest[0].Name =` Mr Jelly
Mr Tall's name:
Mr Nosey's name:
Mr Small's score:

# Arrays of Structures

`StudentRecord[]` `StudentTest`

| | |
|---|---|
| Name: Mr Jelly<br>Score: 3 | 0 |
| Name: Mr Small<br>Score: 87 | 1 |
| Name: Mr Strong<br>Score: 2 | 2 |
| Name: Mr Tall<br>Score: 56 | 3 |
| Name: Mr Nosey<br>Score: 33 | 4 |

```
struct StudentRecord {
  public string name;
  public int score;
}
```

| | |
|---|---|
| `studentTest[2].Name =` | Mr Strong |
| `studentTest[1].Score =` | 87 |
| `studentTest[0].Name =` | Mr Jelly |
| Mr Tall's name: | `StudentTest[3].Name` |
| Mr Nosey's name: | `StudentTest[4].Name` |
| Mr Small's score: | `StudentTest[1].Score` |

# You Try!

On BPCompSci:

## W114 – C# Arrays of Structures